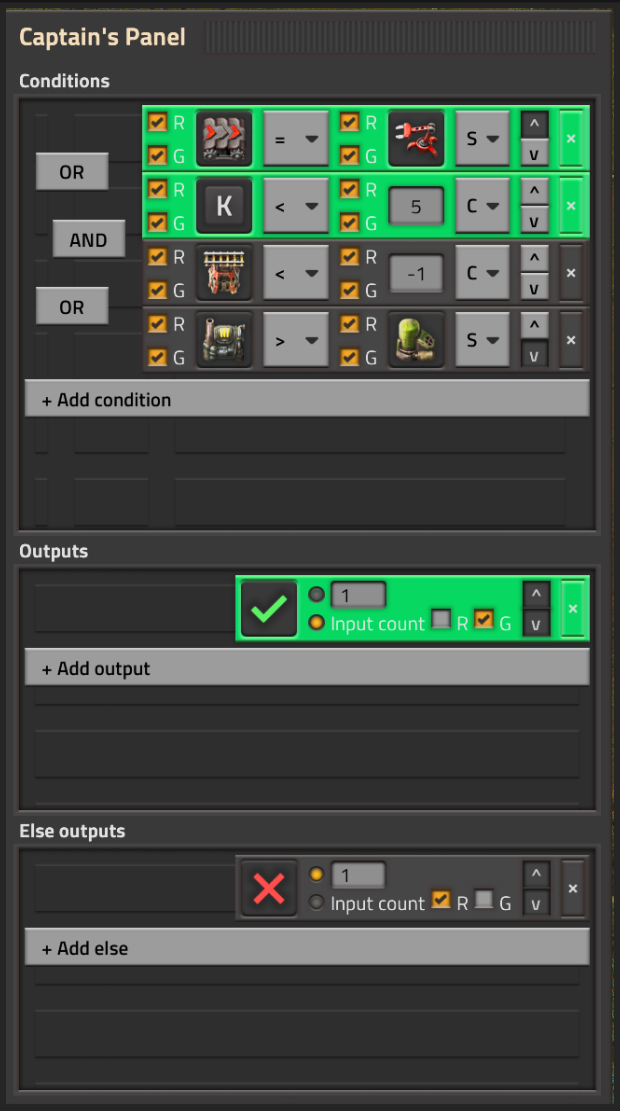
Captain's UI Library

A practical guide for building native-looking Factorio 2.1 mod interfaces with reusable Lua helpers.



Example editor with fulfilled conditions and active normal output.

Purpose

This library helps mods render Factorio-style GUI sections, rows, controls, decider-like conditions, outputs, else outputs, and immediate visual feedback without each mod rebuilding the same foundation.

Contents

> ## Version target
>
> The guide assumes Factorio 2.1 and the current Captain's UI Library control-stage API.

Quick Start

Add the UI library as a dependency, then require the public API from your mod's control stage.

```
local ui = require("__captainjhory-ui-library__.scripts.ui")
```

For the decider editor, store one state table per player, entity, or editor instance. The UI should be rebuilt from that state whenever the player changes a value.

```
local state = ui.decider_state.new_state()

ui.decider_editor.add(parent_element, state, {
  red = {
    ["item/iron-ore/normal"] = 20,
  },
  green = {
    ["virtual/signal-S/normal"] = 1,
  },
})
```

Important

The GUI is not the source of truth. The state table is. The GUI is only a visual rendering of state, plus tagged controls that tell event handlers what changed.

Panel Placement

The decider editor can be rendered anywhere a consuming mod can add GUI children. The library provides two frame helpers for the common cases: anchored relative panels and floating screen windows.

Relative to a native GUI

Use a relative panel when your UI should appear beside a supported Factorio window, such as a reactor, decider combinator, constant combinator, container, or the controller inventory.

```
        ui.relative_panel.open_relative_panel(player, {
          name = "my_mod_reactor_panel",
          caption = { "my-mod.reactor-panel-title" },
          gui = defines.relative_gui_type.reactor_gui,
          position = defines.relative_gui_position.right,
        }, function(frame)
          ui.decider_editor.add(frame, state, signal_values)
        end)
```

Floating screen window

Use a floating panel when the editor should be its own draggable window. The helper creates a frame directly under player.gui.screen and auto-centers it unless you pass a location.

```
        ui.relative_panel.open_floating_panel(player, {
          name = "my_mod_decider_window",
          caption = { "my-mod.decider-window-title" },
        }, function(frame)
          ui.decider_editor.add(frame, state, signal_values)
        end)
```

Rebuild behavior

Both panel helpers destroy an existing frame with the same name before adding a fresh one. That makes immediate UI rebuilds predictable after tagged GUI events.

Core Modules

+ **scripts.ui**: public entry point that exports the library modules.
+ **decider_state**: creates, normalizes, evaluates, moves, and serializes editor state.
+ **decider_editor**: handles GUI events and mutates state based on element tags.
+ **components**: renders the native-looking rows, buttons, backgrounds, and sections.
+ **relative_panel**: opens panels relative to native game GUIs or as floating screen windows.
+ **mod_primitives**: small reusable styling/layout primitives.

Recommended ownership

A consuming mod should own its domain data. The library owns rendering helpers and generic editor state tools. When the player confirms, closes, or when your mod needs to run logic, serialize the editor state into your mod's own data shape.

```
local serialized = ui.decider_state.serialize_decider_editor(state)
-- Store or translate `serialized` into your mod's domain settings.
```

Decider Editor State

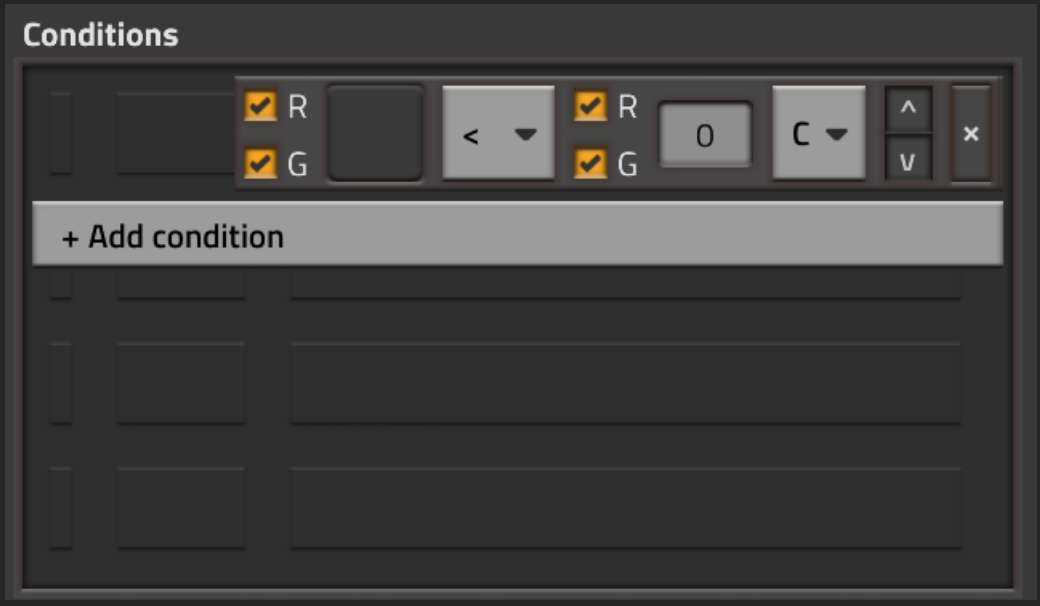
The editor state contains three row collections: conditions, outputs, and else_outputs. It also stores evaluated flags such as condition.fulfilled, output.fulfilled, and state.conditions_fulfilled.

```
local state = {
  conditions_fulfilled = false,
  conditions = { ... },
  outputs = { ... },
  else_outputs = { ... },
}
```

Normalization

Call normalize_state when reading old saved data or accepting partially-built tables. It fills missing defaults, ensures each section has at least one row, and protects the renderer from nil-heavy edge cases.

```
state = ui.decider_state.normalize_state(state)
```



A condition section with one inactive row and native-looking empty background slots.

Rendering And Events

Build the visible editor from state. Controls use tags so event handlers can identify the section, row index, and field that changed.

```
ui.decider_editor.add(parent, state, signal_values)
```

After a handled event changes the editor structure, destroy and rebuild the editor content. This keeps indexes, rows, dropdown choices, and signal picker/display modes synchronized.

```
local changed = ui.decider_editor.handle_click(state, event)
if changed then
  rebuild_editor_for_player(player)
end
```

Why rebuild?

Factorio GUI elements are not automatically bound to your Lua state. Rebuilding is the simple, reliable way to keep structural UI changes honest after add, remove, move, dropdown, text, or signal selection changes.

Signal slot editing

Selected signals render as numbered sprite buttons. Click a selected signal to rebuild that slot as a choose-element button, pick the new signal, then rebuild back to display mode.

Frequent value refresh

When only circuit values changed, do not rebuild. Refresh updates existing tagged GUI elements in place, so an open choose-element picker stays open.

```
ui.decider_editor.refresh(parent, state, signal_values)
```

Inactive condition row.

Fulfilled condition row.

Conditions

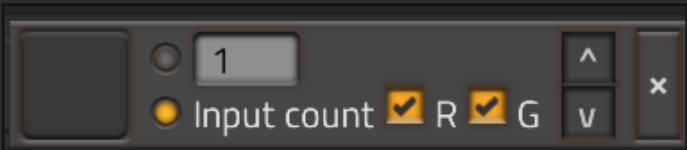
A condition row compares a left signal value against either a constant or another signal. The left and right red/green checkboxes are separate input filters.

+ **left_signal**: selected signal on the left side.
+ **comparator_index**: selected comparison operator.
+ **right_operand_type_index**: constant or signal.
+ **right_constant** or **right_signal**: right side value source.
+ **left_red_enabled** and **left_green_enabled**: left-side input wire filters.
+ **right_red_enabled** and **right_green_enabled**: right-side input wire filters.
+ **joiner**: and/or grouping with the previous row.

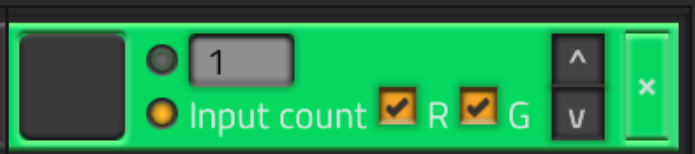
Joiners are placed between condition rows. Green rows are currently fulfilled.

Outputs And Else Outputs

Outputs are active when the whole condition expression is true. Else outputs are active when it is false. Both row types share the same output data shape.



Inactive output row.



Active output row.

Output modes

+ **constant**: output the selected signal with the typed constant.
+ **input_count**: output the selected signal with the count read from enabled input wires.

Example editor with else output active because the full condition expression is false.

Evaluation

Pass signal values as separate red and green tables. The evaluator sums only the wire tables enabled by each row's checkboxes.

```
local signal_values = {
  red = {
    ["virtual/signal-T/normal"] = 500,
  },
  green = {
    ["item/uranium-fuel-cell/normal"] = 2,
  },
}

state = ui.decider_state.evaluate_decider_editor(state, signal_values)
```

Signal keys

The evaluator accepts keys with quality, without quality, or with colon syntax. Prefer type/name/quality, for example item/iron-ore/normal.

AND and OR grouping

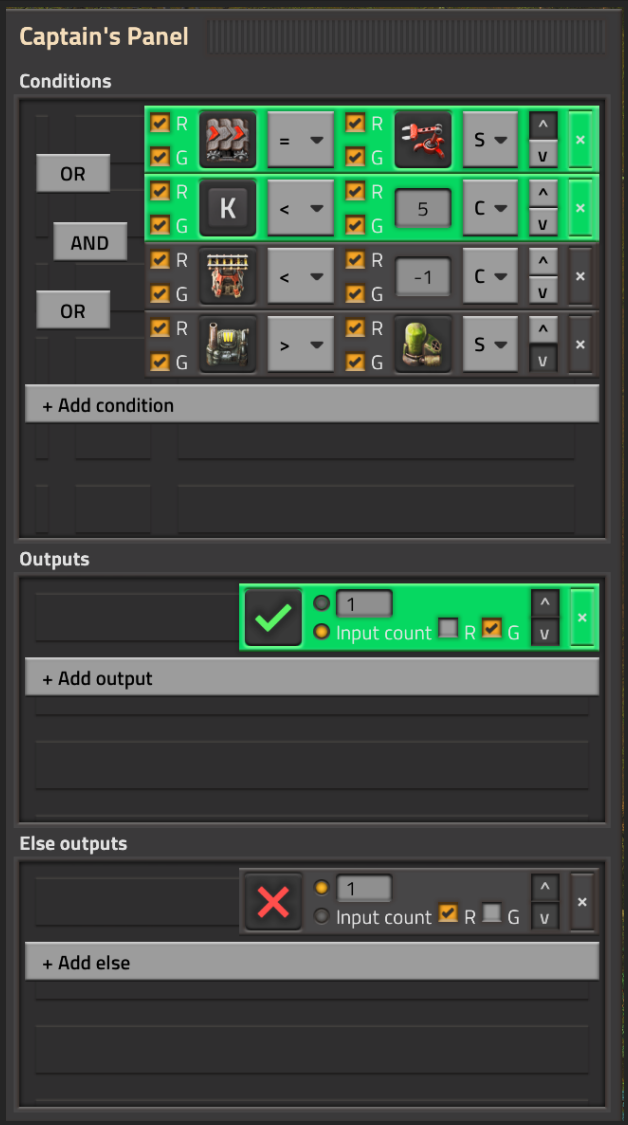
Consecutive AND rows form a group. OR starts a new group. The whole condition expression is true when any group is true.

Serialization

Serialization converts editor state into a smaller data table that is easier for a consuming mod to store or translate into its own runtime logic.

```
local condition_data = ui.decider_state.serialize_condition(condition)
local output_data = ui.decider_state.serialize_output(output)
local editor_data = ui.decider_state.serialize_decider_editor(state)
```

Use serialized data when your mod needs to save configuration, sync entity settings, or convert the editor into behavior. Do not use serialization just to render the UI; render from normalized editor state.



Full editor overview with active normal output.

Production Checklist

+ Keep state in **storage**, keyed by player, entity unit number, or your mod's editor id.
+ Normalize old state after migrations or when loading incomplete data.
+ Rebuild the editor immediately after meaningful GUI events.
+ Evaluate with current red and green signal tables before rendering fulfilled state.
+ Serialize only when handing data to your mod's own logic layer.
+ Use locale keys for visible text in public mods.
+ Keep the built-in demo disabled unless actively developing the library.

Design goal

The best modded UI should feel like it belongs in Factorio. Match native spacing, use compact rows, keep controls immediate, and avoid apply buttons unless the native interaction would use one.